100 Principles Of PromptEngine

Prompt Engineering 的 100 个基本原则

这不是一份“万能提示词合集”，而是一套关于如何与大模型沟通、约束、验证、迭代与避坑的系统化原则地图。

哲学观 | 核心原则 | 思维模型 | 关键方法 | 避坑指南

目录

01

哲学观

与异种智能对话
理解模型本质
建立正确预期

02

核心原则

如何给上下文
如何写约束
如何控格式

03

思维模型

CoT / ToT / RAG
ReAct / Meta-Prompting
Critic / RLHF

04

关键方法

框架化写法
任务拆解
结构化输出

05

避坑指南

提示注入
上下文污染
温度与幻觉

讲法建议： 不要试图逐条念完 100 条。更适合的讲法是先讲核心洞察，再按 5 个主题给出“原则簇”，最后把 100 条当成一张可查阅地图。

原图总览

参考图本身是一张“总地图海报”：上半区给出提示工程的定义与核心洞察，下半区按五个主题各列 20 条原则， 一共构成 100 条基础规则。它非常适合拿来查阅，但并不适合直接投屏讲解。

它讲了什么

提示工程不是“魔法咒语”，而是一门围绕概率模型展开的人机沟通艺术。

它怎么组织

哲学观、核心原则、思维模型、关键方法、避坑指南，五个主题共同组成全景图。

这次怎么讲

把海报式信息重构成可讲的章节、卡片和流程，让听众先懂，再记住，再能复用。

第一部分：哲学观

与异种智能对话

先理解模型是什么，再谈怎么写提示词。

哲学观：前 20 条原则

01 - 10

01. GIGO 原则
02. 概率本质
03. 自然语言编程
04. 人机协同
05. 上下文即记忆
06. 压缩与解压
07. 幻觉是特性
08. 对齐（Alignment）
09. 没有魔法
10. 迭代至上

11 - 20

11. 模型不可知论
12. Token 思维
13. 注意力机制
14. 少即是多
15. 清晰度
16. 显性化
17. 动态演进
18. 黑盒摸索
19. 伦理边界
20. 工具属性

哲学观：你真正要记住的 4 件事

模型不是专家，是概率引擎

它的默认动作不是“理解你”，而是“接着写最可能的内容”。

Prompt 不是咒语，是约束说明书

越清晰、越显式、越结构化，越容易收敛到想要的输出。

上下文就是工作记忆

窗口里有什么，模型就更像什么；窗口外的东西，它未必记得住。

最强动作不是“神 prompt”，而是迭代

先得到一个可工作的结果，再不断加约束、加例子、加评估。

第二部分：核心原则

刚性约束

Prompt 的主要工作，不是“多说”，而是“把边界讲清楚”。

核心原则：21 - 40

21 - 30

21. 具体（Specific）
22. 背景（Context）
23. 角色（Persona）
24. 约束（Constraint）
25. 示例（Example）
26. 格式（Format）
27. 步骤（Steps）
28. 无歧义
29. 正向指令
30. 分隔符

31 - 40

31. 引用来源
32. 自我修正
33. 输出长度
34. 温度控制
35. 版本管理
36. 模块化
37. 语言一致
38. 拒绝回答策略
39. 语气设定
40. 结束语

核心原则：一个高质量 Prompt 的骨架

背景

我要解决什么问题

角色

你扮演谁，用什么视角

任务

具体要输出什么

约束

不要做什么，做到什么粒度

格式

用什么结构返回

最实用的经验不是背更多技巧，而是先问自己：**我的 Prompt 里有没有背景、边界、例子、格式和验收标准？**

第三部分：思维模型

认知工具

这些不是“花哨技巧”，而是让模型更稳定推理的工作模式。

思维模型：41 - 60

41 - 50

41. 实习生模型
42. 自动补全机
43. 思维链（CoT）
44. 零样本（Zero-shot）
45. 少样本（Few-shot）
46. ToT（Tree of Thoughts）
47. RAG
48. ReAct
49. 多角色辩论
50. 反向提示

51 - 60

51. 分治策略
52. 漏斗模型
53. 元提示（Meta-Prompting）
54. 三明治结构
55. 模拟器思维
56. 翻译层思维
57. 潜空间（Latent Space）
58. 批判者（Critic）
59. 模板化
60. 人类反馈（RLHF）

思维模型：最值得掌握的 6 个

CoT

让模型显式展示思考步骤，适合数学、逻辑与复杂分析。

Few-shot

通过示例建立输出风格和格式，是最快速的“对齐器”。

RAG

先找资料再回答，适合知识更新快、需要引用依据的任务。

ReAct

Reasoning + Acting，让模型思考后调用工具，适合 Agent 工作流。

Critic

引入自我批评或第二视角，能显著提高答案稳健性。

Meta-Prompting

让 AI 帮你写 Prompt，本质是把提示工程本身变成任务。

第四部分：关键方法论

如何行动

从框架到流程，从提问方式到输出控制，这部分最像可直接上手的工具箱。

关键方法论：61 - 80

61 - 70

61. CRISPE 框架
62. BROKE 框架
63. 逐步细化
64. AB 测试
65. 长文总结
66. 风格模仿
67. 数据提取
68. 代码解释
69. 苏格拉底提问
70. 自我一致性

71 - 80

71. 工具调用
72. 提示压缩
73. 长上下文利用
74. 情绪勒索
75. 打破回音室
76. 结构化前缀
77. 递归提示
78. 降维打击
79. 跨语言桥接
80. 保存预设

关键方法论：一个好用的执行套路

框架化

用 CRISPE、BROKE 这种骨架降低遗漏概率。

拆解

复杂任务不要一步到位，先拆步骤，再逐段校验。

结构化输出

优先要求 JSON、表格、列表，让结果可复用、可比较。

测试与迭代

AB 测试、长上下文回放、自我一致性，是调 Prompt 的基本工。

越重要的任务，越不该靠“一次性灵感 Prompt”。真正稳定的是框架、流程和校验机制。

第五部分：避坑指南

生存法则

好的 Prompt 不只会“让模型更会做事”，还要“让模型不容易做错事”。

避坑指南：81 - 100

81 - 90

81. 警惕提示注入
82. 不要用否定句
83. Token 截断
84. 过拟合
85. 复杂指令过载
86. 逻辑陷阱
87. 数学不行
88. 最新实事
89. 隐私泄露
90. 盲信权威

91 - 100

91. 上下文污染
92. 指令冲突
93. 测试不足
94. 忽视微调
95. 滥用 CoT
96. 格式漂移
97. 过度依赖
98. 忽视温度
99. 安全过滤器
100. 停止思考

最常见的 5 个翻车点

注入与污染

输入里混入恶意指令、旧上下文干扰当前任务，是 Agent 时代最危险的基础风险。

负向表达与冲突

“不要像大象”这类写法容易误导模型；System Prompt 与 User Prompt 冲突更会直接失真。

超长与截断

太长、太杂、太多附件，常常不是更强，而是让关键约束被挤出窗口。

过度相信推理

涉及数学、最新知识、复杂逻辑时，最好让模型调工具或给出可验证依据。

没有评测，就没有工程

Prompt 工程的下半场不是写，而是测。只拿一个例子感觉“好像行了”，通常迟早翻车。

把 100 条原则压缩成一页工作流

1. 定义任务

背景、目标、角色、约束先讲清楚。

2. 给结构

格式、步骤、示例、输出模版写明白。

3. 选模式

Few-shot、CoT、RAG、ReAct，按任务选模型。

4. 跑测试

AB 测试、边界样例、长上下文回放。

5. 做防护

注入、冲突、截断、隐私、安全过滤一起看。

一句话结论

Prompt Engineering 不是“凑几个高能词”，而是把任务、约束、思维模式、工具和评测机制组合成一套稳定的人机协作系统。

最后一句

提示工程的终点，不是让模型替你思考，
而是让你的意图、边界和验证方式，
被模型稳定地理解和执行。